TECNOLOGIA SETÚBAL
ESCOLA SUPERIOR POLITÉCNICO SETÚBAL

# Programação Avançada

## Licenciatura em Engª. Informática

## Relatório Técnico

**Docente**: André Sanguinetti
Nº202200187, Gonçalo Barracha
Nº202200197, Rodrigo Cardoso
Nº202200186, Francisco Paz

# 1. Introdução

O projeto de **Sistema Integrado de Transportes** propõe o desenvolvimento de uma solução eficiente para interligar diversos meios de transporte público, permitindo a deslocação integrada e otimizada de pessoas ao longo de uma rede urbana ou interurbana. Esta solução visa facilitar o acesso e a utilização combinada de diferentes modalidades, como comboios, autocarros, barcos, bicicletas e percursos a pé, promovendo uma mobilidade mais sustentável e inclusiva.

A estrutura do sistema baseia-se na modelação da rede de transporte através de um grafo, onde as **paragens** são representadas por vértices, denominados **stops**, e as **rotas** entre elas por arestas, denominadas **routes**. Este modelo bidirecional permite não apenas mapear a rede, mas também identificar as melhores opções de percurso entre dois pontos, mesmo que envolvam múltiplos meios de transporte.

Com foco na eficiência, na acessibilidade e na simplicidade de utilização, o projeto tem como objetivo principal melhorar a experiência do utilizador, promovendo deslocações mais rápidas, cómodas e alinhadas com as necessidades modernas de mobilidade sustentável. Este sistema pode beneficiar tanto os utilizadores regulares quanto as cidades, reduzindo congestionamentos, emissões de carbono e promovendo um transporte coletivo mais integrado e ecológico.

# 2. Tipos Abstratos de Dados

O projeto utiliza várias Abstract Data Types (ADTs), principalmente para representar grafos e seus elementos (paragens, rotas). Abaixo está a descrição das interfaces Java e as implementações associadas.

**Grafo (ADT Graph)**
*Interface Java:*
A estrutura de grafo é representada pela interface Graph<V, E>.

*Métodos da interface:*
*Manipulação de Vértices e Arestas:*
insertVertex(V element): Adiciona um vértice ao grafo.
insertEdge(V v1, V v2, E element): Adiciona uma aresta entre dois vértices.
removeVertex(Vertex<V>): Remove um vértice do grafo.
removeEdge(Edge<E, V>): Remove uma aresta do grafo.
*Consulta de Elementos:*
vertices(): Retorna todos os vértices do grafo.
edges(): Retorna todas as arestas do grafo.
incidentEdges(Vertex<V>): Obtém as arestas incidentes a um vértice.
opposite(Vertex<V>, Edge<E, V>): Retorna o vértice oposto em uma aresta.

*Implementação:*
A classe TransportMap implementa o grafo estendendo GraphEdgeList<Stop, Route>.
*Estrutura de Dados Interna:*
Lista de Vértices: Representa as paragens como instâncias de Vertex<Stop>.
Lista de Arestas: Representa as rotas como instâncias de Edge<Route, Stop>.

*Métodos Adicionais em TransportMap:*
*Centralidade:*
getCentralityList(): Calcula a centralidade das paragens com base no número de conexões.
getTop5CentralStops(): Obtém as 5 paragens mais conectadas.
Caminho Acessível:
getAccessibleStops(String stopName, int distance): Obtém todas as paragens acessíveis dentro de uma distância máxima.
Busca e Consulta:
findStopByCode(String code): Busca uma paragem pelo código.
findVertexByName(String stopName): Busca um vértice pelo nome da paragem.

**Memento (ADT Memento)**
*Interface Java:*
A interface Memento é utilizada para implementar o padrão Memento.

*Métodos da interface:*
A interface em si não define métodos, mas a sua funcionalidade depende do Originator.

*Implementação:*
A classe MapViewMemento encapsula o estado do grafo.
*Atributos:*
edgeList: Uma coleção das arestas no estado salvo.
*Construtor:*
MapViewMemento(Collection<Edge<Route, Stop>> edgeList): Cria um memento a partir de uma lista de arestas.

**Originator (ADT Originator)**
*Interface Java:*
Interface que define os métodos para criar e restaurar mementos.

*Métodos da interface:*
createMemento(): Cria um novo memento com o estado atual.
setMemento(Memento memento): Restaura o estado a partir de um memento.

*Implementação:*
A classe MapView implementa Originator:
*Criação de Memento:*

createMemento(): Salva o estado atual das arestas do grafo.
*Restaurar Estado:*
setMemento(Memento memento): Restaura o estado do grafo usando o memento fornecido.

**Estratégias de Cálculo de Custo (ADT CostCalculationStrategy)**
*Interface Java:*
Define a lógica para calcular custos em arestas.

*Métodos da interface:*
calculateCost(Edge<Route, Stop> edge, String transportType): Retorna o custo associado a uma aresta e tipo de transporte.

*Implementações:*
DistanceStrategy: Baseado na distância da rota.
DurationStrategy: Baseado na duração da rota.
SustainabilityStrategy: Baseado na sustentabilidade ($CO_2$).

**Logger (ADT Singleton)**
*Interface Java:*
Singleton que registra eventos no sistema.

*Métodos da interface:*
log(String message): Escreve uma mensagem no arquivo de log.

*Implementação:*
*Classe Logger:*
*Atributos:*
LOG_FILE_PATH: Caminho do arquivo de log.
instance: Instância única do logger.
*Métodos:*
log(String message): Escreve uma mensagem no log com timestamp.

## 3. Diagrama de classes

## 4. Padrões de Software

**Padrão Factory**

*Descrição:* O padrão Factory é usado para criar objetos sem expor a lógica de criação ao cliente, delegando essa responsabilidade a subclasses ou classes específicas.

*Razão de Seleção:* A necessidade de criar diferentes implementações do cálculo de custo (CostCalculationStrategy) sem sobrecarregar a lógica de criação em outras partes do código. Facilita a extensão, permitindo a adição de novas estratégias sem modificar o código existente.

*Mapeamento no Projeto:*
*Interface StrategyFactory:* Define o contrato para a criação de estratégias.
*Classe CreateStrategyFactory:* Implementa a lógica para criar instâncias concretas de estratégias (DistanceStrategy, DurationStrategy, SustainabilityStrategy).

**Padrão Strategy**

*Descrição:* O padrão Strategy define uma família de algoritmos, encapsula cada um deles, e os torna intercambiáveis. Ele permite variar o algoritmo independentemente dos clientes que o utilizam.

*Razão de Seleção:* A necessidade de calcular custos de forma flexível, com base em diferentes critérios (distância, duração, sustentabilidade), sem alterar o comportamento principal do algoritmo.

*Mapeamento no Projeto:*
*Interface CostCalculationStrategy:* Define o contrato para estratégias de cálculo de custo.

*Implementações*: **DistanceStrategy** para cálculo baseado em distâncias. **DurationStrategy** para cálculo baseado em duração. SustainabilityStrategy para cálculo baseado em sustentabilidade.

**Padrão Memento**
*Descrição:* O padrão Memento captura e armazena o estado interno de um objeto sem violar o encapsulamento, para que ele possa ser restaurado posteriormente.

*Razão de Seleção:* Necessidade de implementar um sistema de "Undo" para o grafo (e.g., ao desativar uma rota). Evita que detalhes internos do estado do grafo sejam expostos diretamente.

*Mapeamento no Projeto:*
*Interface Memento:* Representa o estado salvo do grafo.
*Interface Originator:* Define os métodos para criar e restaurar estados (createMemento e setMemento).
*Classe MapView:* Implementa Originator e salva/restaura o estado do grafo visual. O estado do grafo é armazenado como uma lista de arestas em MapViewMemento.
*Classe MapViewCaretaker:* Gerencia os estados salvos e fornece funcionalidade de restauração.

**Padrão Singleton**
*Descrição:* O padrão Singleton assegura que uma classe tenha apenas uma instância e fornece um ponto de acesso global a essa instância.

*Razão de Seleção:* Necessidade de centralizar funcionalidades que são usadas em várias partes do sistema, como logging e acesso a propriedades de configuração.

*Mapeamento no Projeto:*
*Classe Logger:* Fornece uma única instância para registar logs de eventos do sistema. O método estático getInstance garante que apenas uma instância de Logger seja criada.
*Classe PropertiesUtil:* Centraliza o carregamento e acesso às configurações do projeto a partir de um arquivo .properties. Usa getInstance para garantir que as propriedades sejam carregadas apenas uma vez.

**Padrão Observer**
*Descrição:* O padrão Observer estabelece uma dependência "um-para-muitos" entre objetos, onde mudanças no estado de um objeto (o sujeito/observável) são automaticamente notificadas para outros objetos dependentes (os observadores). Ele permite que os observadores sejam atualizados sem que o sujeito precise saber detalhes sobre eles.

*Razão de Seleção*: No contexto deste projeto, o padrão Observer foi selecionado para resolver o problema de sincronização entre o modelo de grafo (TransportMap) e sua visualização na interface gráfica (MapView).

*Mapeamento no Projeto:*
*Interface Observable:* Define os métodos para gerenciar observadores
*Interface Observer:* Define o método update(), chamado quando o sujeito notifica os observadores.
*Classe TransportMap (Sujeito/Observable):* O TransportMap implementa Observable e notifica os observadores quando um vértice ou aresta é adicionado ou removido. O método notifyObservers() é chamado.
*Classe MapView (Observador):* O MapView implementa Observer e reage às notificações do TransportMap. Ele chama graphView.update() para refletir as mudanças no grafo visual.

## 5. Refactoring

| Code Smells | Técnica Refactoring | Número situações |
|---|---|---|
| Duplicate Code | Extract Method | 5 |
| Hard Coding | Extract Constant | 3 |
| Missing Validation | Null Check/Error handling | 6 |
| Large Class | Single Responsibility Principle | 3 |
| UI Design | Extract Method/UI Factory | 1 |
| Long Method | Extract Method | 5 |
| Tight Coupling | Dependency Injection | 3 |
| High Cyclomatic Complexity | Extract Method | 4 |
| Excessive Responsibility | Single Responsibility Principle | 2 |
| Complex Conditional Logic | Replace Nested Conditional with Guard Clauses | 3 |
| Primitive Obsession | Replace Primitive with Object | 4 |
| Lack of Encapsulation | Encapsulate Field | 2 |
| Inconsistent Naming | Rename Method | 7 |

## 6. Conclusão

Este projeto foi uma excelente oportunidade para consolidarmos os conhecimentos adquiridos ao longo do semestre, permitindo-nos aplicar a teoria de forma prática e desafiadora. Através da modelação de um sistema de transporte integrado e da implementação de diversos padrões de software, conseguimos compreender melhor a matéria lecionada e explorar novas formas de resolver problemas complexos.